04. Divide-and-Conquer

Hyunjong Choi, Ph.D.
Assistant Professor of Computer Science
San Diego State University

Outline

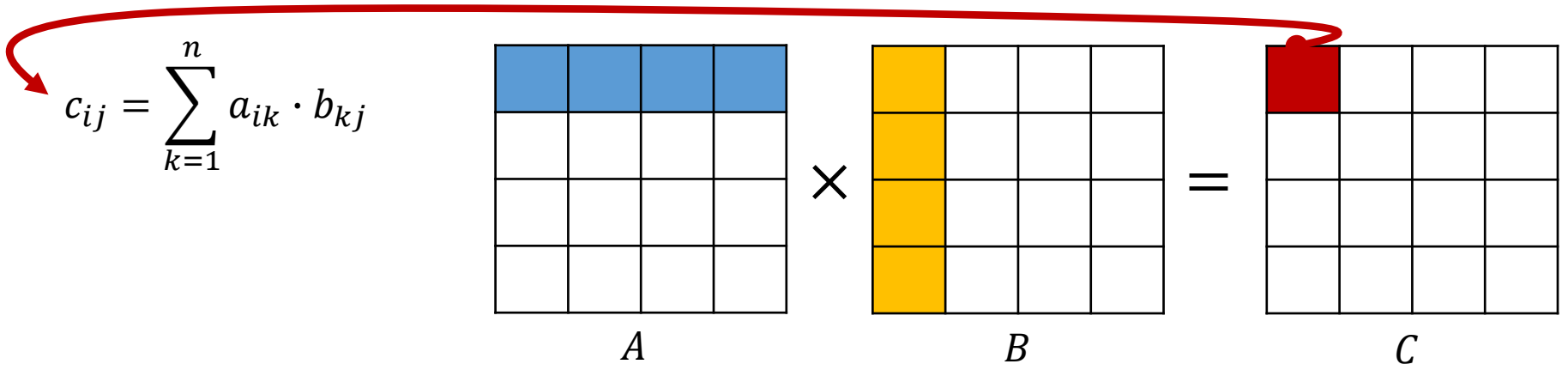
- More algorithms based on ***divide-and-conquer***
  - Maximum-subarray problem (Textbook 3rd edition 68 – 74)
  - Matrix multiplication problem (straightforward method vs. Strassen's algorithm) (Textbook 3rd edition 75 – 82)

- Solving recurrence equations
  - Substitution method for solving recurrence
  - Recursion-tree method
  - Master method (master theorem)

Matrix multiplication problem

- Use the divide-and-conquer method to multiply ***square*** matrices.
- Multiplying square matrices
  - $A = (a_{ij})$ and $B = (b_{ij})$ are square $n \times n$ matrices
  - Then in the product $C = A \cdot B = (c_{ij})$, the entry c_{ij} is defined by:

$$c_{ij} = \sum_{k=1}^{n} a_{ik} \cdot b_{kj}$$



  - Computing n^2 matrix entries, each of which is the sum of n pairwise products of input elements from A and B

Naïve approach square matrix multiplication

❑ Pseudocode for SQUARE-MATRIX-MULTIPLY(A, B)

```
SQUARE-MATRIX-MULTIPLY(A, B)
1.  n = A.rows
2.  let C be a new n × n matrix
3.  for i = 1 to n
4.      for j = 1 to n
5.          c_ij = 0
6.          for k = 1 to n
7.              c_ij = c_ij + a_ik · b_kj
8.  return C
```

- Compute entries in each of n rows
- Compute n entries in row i
- Compute the sum of n items like $c_{ij} = \sum_{k=1}^{n} a_{ik} \cdot b_{kj}$

- Each of the triply-nested **for** loops runs exactly n iterations

➡ The running time of the procedure is $\boldsymbol{\Theta(n^3)}$

Divide-and-conquer approach

- Assume that n is a power of 2 in each of the $n \times n$ matrices.
- **Divide step**: partitions $n \times n$ matrices into **four** $n/2 \times n/2$ submatices.

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

- We can rewrite the equation $C = A \cdot B$ as

Eight $\frac{n}{2} \times \frac{n}{2}$ multiplications & **four** additions of $\frac{n}{2} \times \frac{n}{2}$ submatrices

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \Rightarrow$$

$$\begin{aligned} C_{11} &= A_{11} \cdot B_{11} + A_{12} \cdot B_{21} \\ C_{12} &= A_{11} \cdot B_{12} + A_{12} \cdot B_{22} \\ C_{21} &= A_{21} \cdot B_{11} + A_{22} \cdot B_{21} \\ C_{22} &= A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \end{aligned}$$

Divide-and-conquer approach (cont.)

- **Conquer** and **combine** steps with recursive approach
- Pseudocode of SQUARE-MATRIX-MULTIPLY-RECURSIVE

```
SQUARE-MATRIX-MULTIPLY-RECURSIVE(A, B)
1.  n = A.rows
2.  let C be a new n × n matrix
3.  if n == 1
4.      c11 = a11 · b11
5.  else partition A, B, and C
6.      C11 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A11, B11)
            + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A12, B21)
7.      C12 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A11, B12)
            + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A12, B22)
8.      C21 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A21, B11)
            + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A22, B21)
9.      C22 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A21, B12)
            + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A22, B22)
10. return C
```

Divide

Conquer and combine

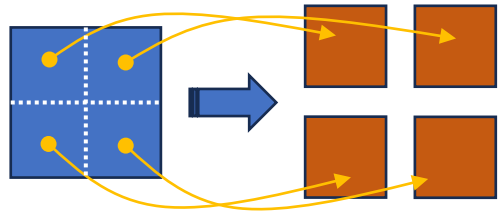
Divide step: how to partition ?

❑ Two methods for partitioning matrix

```
SQUARE-MATRIX-MULTIPLY-RECURSIVE(A, B)
1.  n = A.rows
2.  let C be a new n × n matrix
3.  if n == 1
4.      c11 = a11 · b11
5.  else partition A, B, and C
6.      C11 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A11, B11)
            + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A12, B21)
7.      C12 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A11, B12)
            + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A12, B22)
8.      C21 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A21, B11)
            + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A22, B21)
9.      C22 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A21, B12)
            + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A22, B22)
10. return C
```

- ***Method 1.*** Allocate temporary storage to hold $A, B,$ and C's four submatrices, e.g., 12 new $\frac{n}{2} \times \frac{n}{2}$ matrices.

  ➡ It will cost $\boldsymbol{\Theta(n^2)}$ time.



- ***Method 2.*** Calculate index to represent a submatrix by a range of row and column indices of the original matrix.

  ➡ It will cost $\boldsymbol{\Theta(1)}$ time.

e.g., Row of A_{11} is $1\ to\ \frac{n}{2}$, Column of A_{11} is $1\ to\ \frac{n}{2}$

Running time analysis

❑ Running time of SQUARE-MATRIX-MULTIPLY-RECURSIVE(A, B)

```
SQUARE-MATRIX-MULTIPLY-RECURSIVE(A, B)
1.  n = A.rows
2.  let C be a new n × n matrix
3.  if n == 1
4.      c11 = a11 · b11
5.  else partition A, B, and C
6.      C11 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A11, B11)
          + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A12, B21)
7.      C12 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A11, B12)
          + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A12, B22)
8.      C21 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A21, B11)
          + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A22, B21)
9.      C22 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A21, B12)
          + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A22, B22)
10. return C
```

Partition takes $\Theta(1)$ time

- **Base case (n == 1)**: perform one multiplication in line 4.

  ➡ $T(1) = \Theta(1)$

$T(n/2)$

$T(n/2)$

~~$8T(n/2)$~~

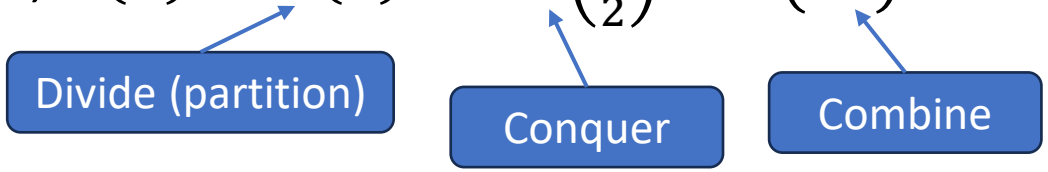
➡ $8T(n/2) + \Theta(n^2)$

- What's amount of time for adding two $\frac{n}{2} \times \frac{n}{2}$ matrices?

  ➡ • Each matrix contains $\frac{n^2}{4}$ entries, thus, $\Theta(n^2)$

Running time analysis (cont.)

- For $n > 1$, $T(n) = \Theta(1) + 8T\left(\frac{n}{2}\right) + \Theta(n^2) = 8T\left(\frac{n}{2}\right) + \Theta(n^2)$



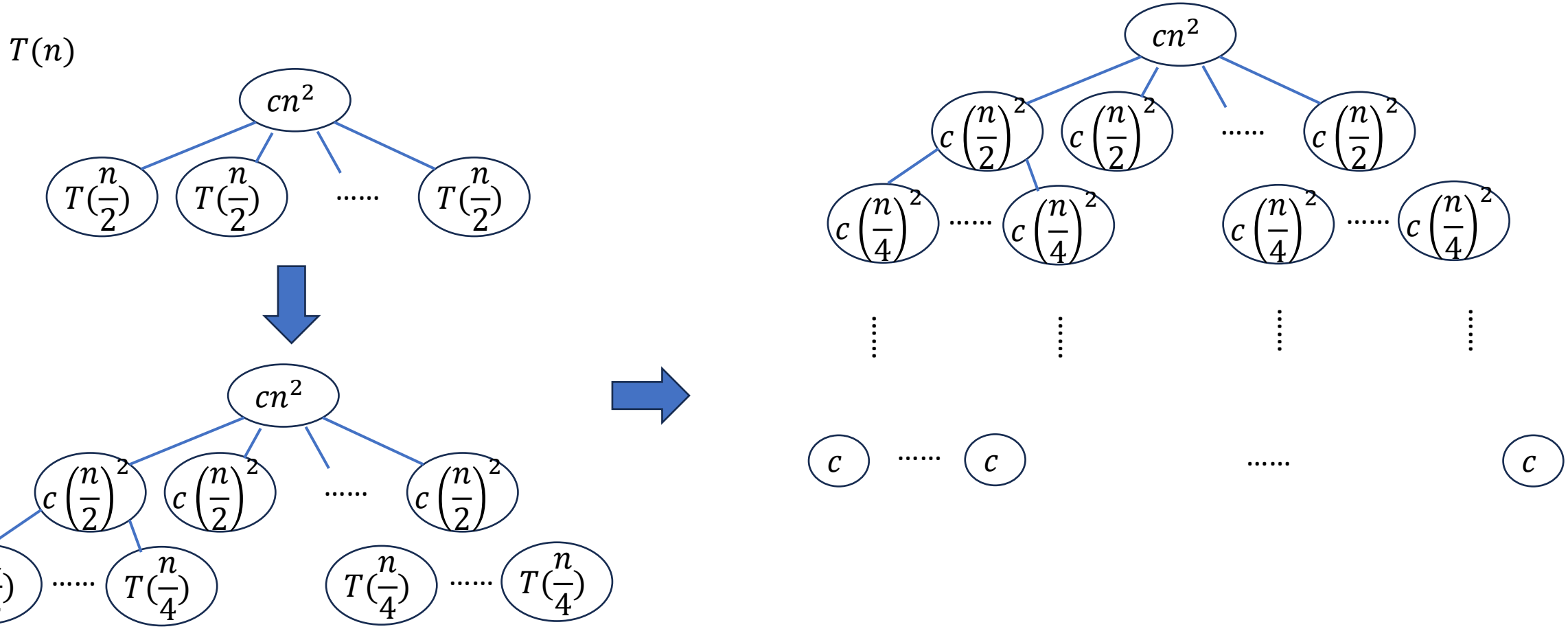
- Putting together, we have

$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1 \\ 8T\left(\frac{n}{2}\right) + \Theta(n^2), & \text{if } n > 1 \end{cases}$$

How to solve this recurrence equation?

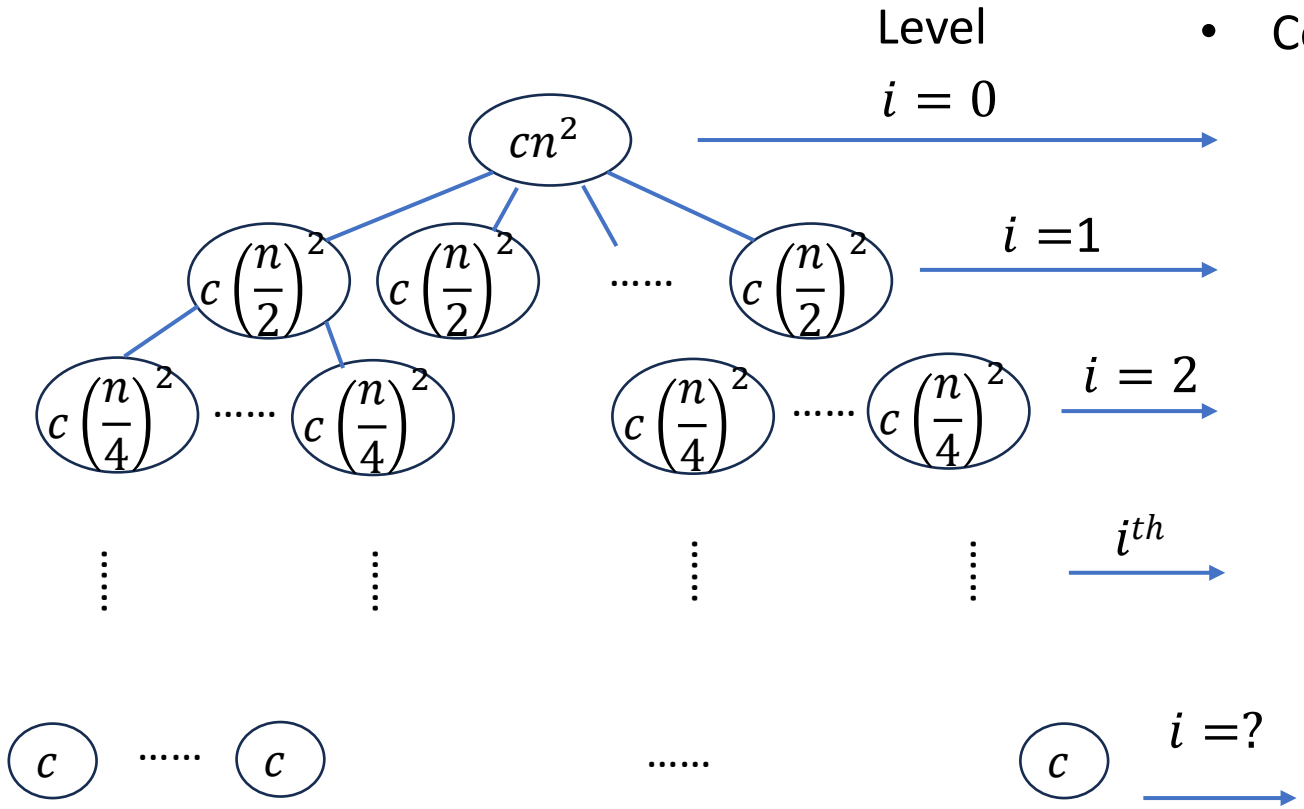
Running time analysis (cont.)

- Recursion tree method



Running time analysis (cont.)

❑ Recursion tree method



- Costs per-level

cn^2

$8 \times c(\frac{n}{2})^2$

$8^2 \times c(\frac{n}{4})^2$

$8^i \times c(\frac{n}{2^i})^2$

$8^i \times c$

- Number of levels

$(\frac{n}{2^i})^2 = 1$

$i = \log_2 n$

- Total cost

$$\sum_{i=0}^{\log_2 n} 8^i c(\frac{n}{2^i})^2$$

$$= 2cn^3 - cn^2$$

$$= \Theta(n^3)$$

Running time analysis (cont.)

- Solve it mathematically
- $T(n) = 8T\left(\frac{n}{2}\right) + \Theta(n^2) = 8T\left(\frac{n}{2}\right) + n^2$

$$= n^2 + 8T\left(\frac{n}{2}\right)$$
$$= n^2 + 8\left(8T\left(\frac{n}{2^2}\right) + \left(\frac{n}{2}\right)^2\right)$$
$$= n^2 + 8^2T\left(\frac{n}{2^2}\right) + 8\left(\frac{n^2}{4}\right)$$
$$= n^2 + 2n^2 + 8^2T\left(\frac{n}{2^2}\right)$$
$$= n^2 + 2n^2 + 8^2\left(8T\left(\frac{n}{2^3}\right) + \left(\frac{n}{2^2}\right)^2\right)$$
$$= n^2 + 2n^2 + 8^3T\left(\frac{n}{2^3}\right) + 8^2\left(\frac{n^2}{4^2}\right)$$
$$= n^2 + 2n^2 + 2^2n^2 + 8^3T\left(\frac{n}{2^3}\right)$$
$$\dots$$
$$= n^2 + 2n^2 + 2^2n^2 + 2^3n^2 + 2^4n^2 + \cdots$$

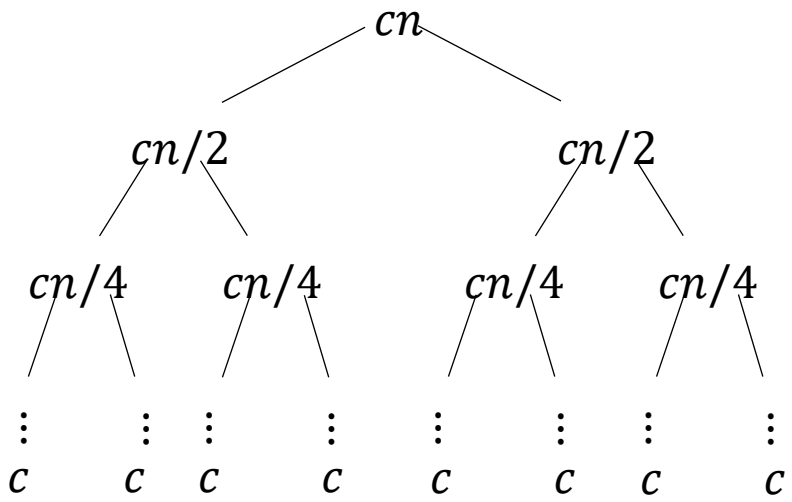
- How long does it continue? where $\frac{n}{2^i} = 1 \Rightarrow i = \log n$
- What is the last term? $8^iT(1) = 8^{\log n}$

$$T(n) = n^2 + 2n^2 + 2^2n^2 + 2^3n^2 + 2^4n^2 + \cdots + 2^{\log n - 1}n^2 + 8^{\log n}$$
$$= \sum_{i=0}^{\log n} 2^in^2 = n^2\sum_{i=0}^{\log n} 2^i = n^2\left(\frac{2^{log2+1} - 1}{2 - 1}\right) = 2n^3 - n^2$$
$$= \Theta(n^3)$$

➡ Same asymptotic running time as the straightforward SQUARE-MATRIX-MULTIPLY procedure.

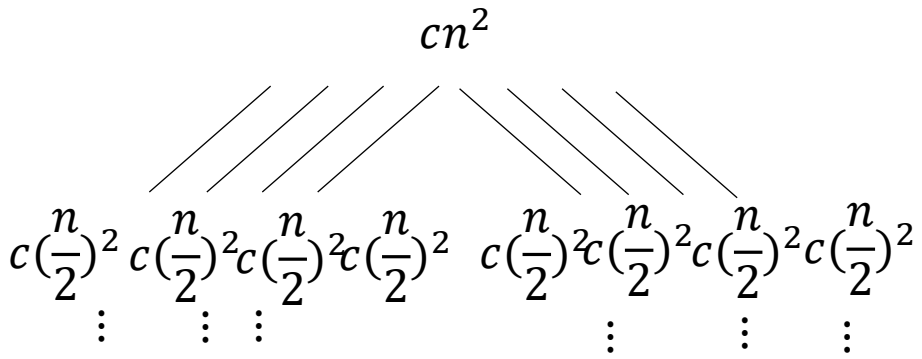
Ideas for improving

- Comparison with merge sort



$$T(n) = \begin{cases} c, & \text{if } n = 1 \\ 2T(n/2) + cn, & \text{if } n > 1 \end{cases}$$

< Merge sort recursion tree >



$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1 \\ 8T\left(\frac{n}{2}\right) + \Theta(n^2), & \text{if } n > 1 \end{cases}$$

By Master method

$$\mathbf{T(n) = \Theta(8^{log_2 n}) = \Theta(n^3)}$$

< Matrix multiplication recursion tree >

➡ Goal: reduce the number of multiplications, i.e., less bushy

Strassen's algorithm

- V. Strassen published a remarkable recursive algorithm for multiplying $n \times n$ matrices in 1969.
  - It makes the recursion tree less bushy: instead of performing eight recursive multiplication of $n/2 \times n/2$ matrices, it performs only **seven**.
  - With the **cost**: several new additions of $n/2 \times n/2$ matrices.

- *Less multiplications, more additions*
  - e.g., $x^2 - y^2$ ⟶ Two multiplications and one addition

Recall old algebra trick: $(x - y)(x + y)$ ⟶ One multiplication and two additions

⇒ If x, y are scalar, there's not much difference. However, if x and y are large matrices, the cost of multiplying outweighs the cost of adding.

Strassen's algorithm (cont.)

- Goal: Make the recursion tree less bushy by performing only **7 recursive multiplications** of $n/2 \times n/2$ matrices.

< Strassen's algorithm >

Step 1. Divide A, B, and C into $n/2 \times n/2$ submatrices just as in SQUARE-MATRIX-MULTIPLY-RECURSIVE. → Takes $\boldsymbol{\Theta(1)}$ time.

Step 2. Create 10 matrices $S_1, S_2, \ldots, S_{10}$, each of which is $n/2 \times n/2$ and is the sum or difference of two matrices created in step 1. → Takes $\boldsymbol{\Theta(n^2)}$ time.

Step 3. Using the submatrices created in step 1 and the 10 matrices created in step 2, recursively compute **seven** matrix products $\boldsymbol{P_1, P_2, \ldots, P_7}$. Each matrix P_i is $n/2 \times n/2$.

Step 4. Compute the desired submatrices C_{11}, C_{12}, C_{21}, C_{22} of the result matrix C by adding and subtracting various combinations of the P_i matrices. → Takes $\boldsymbol{\Theta(n^2)}$ time.

Strassen's algorithm detail

- Step 1. partitions matrices:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

- Step 2. creates the following 10 matrices:

$$S_1 = B_{12} - B_{22}$$
$$S_2 = A_{11} + A_{12}$$
$$S_3 = A_{21} + A_{22}$$
$$S_4 = B_{21} - B_{11}$$
$$S_5 = A_{11} + A_{22}$$
$$S_6 = B_{11} + B_{22}$$
$$S_7 = A_{12} - A_{22}$$
$$S_8 = B_{21} + B_{22}$$
$$S_9 = A_{11} - B_{21}$$
$$S_{10} = B_{11} + B_{12}$$

- Add or subtract $n/2 \times n/2$ matrices 10 times, which takes $\Theta(n^2)$ time.

Strassen's algorithm detail

- Step 3. recursively multiplies $n/2 \times n/2$ matrices 7 times to compute the following matrices:

$$
\begin{aligned}
P_1 &= A_{11} \cdot S_1 = A_{11} \cdot B_{12} - A_{11} \cdot B_{22} \\
P_2 &= S_2 \cdot B_{22} = A_{11} \cdot B_{22} + A_{12} \cdot B_{22} \\
P_3 &= S_3 \cdot B_{11} = A_{21} \cdot B_{11} + A_{22} \cdot B_{11} \\
P_4 &= A_{22} \cdot S_4 = A_{22} \cdot B_{21} - A_{22} \cdot B_{11} \\
P_5 &= S_5 \cdot S_6 = A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22} \\
P_6 &= S_7 \cdot S_8 = A_{12} \cdot B_{21} + A_{12} \cdot B_{22} - A_{22} \cdot B_{21} - A_{22} \cdot B_{22} \\
P_7 &= S_9 \cdot S_{10} = A_{11} \cdot B_{11} + A_{11} \cdot B_{12} - A_{21} \cdot B_{11} - A_{21} \cdot B_{12}
\end{aligned}
$$

- Do not do the computation, just show what the productions equal in terms of the original submatrices created in step 1.

- These are the *actual* computation that we need to perform

Strassen's algorithm detail

- Step 4. adds and subtracts the P_i matrices created in step 3 to construct the four $n/2 \times n/2$ submatrices of C, i.e., C_{11}, C_{12}, C_{21}, and C_{22}.

- $C_{11} = P_5 + P_4 - P_2 + P_6$

$$A_{11} \cdot B_{11} + \cancel{A_{11} \cdot B_{22}} + \cancel{A_{22} \cdot B_{11}} + \cancel{A_{22} \cdot B_{22}}$$
$$\cancel{-A_{22} \cdot B_{11}} \quad \cancel{+A_{22} \cdot B_{21}}$$
$$\cancel{-A_{11} \cdot B_{22}} \quad \cancel{-A_{12} \cdot B_{22}}$$
$$\cancel{-A_{22} \cdot B_{22}} \quad \cancel{A_{22} \cdot B_{21}} \quad \cancel{+A_{12} \cdot B_{22}} \quad +A_{12} \cdot B_{21}$$

$$A_{11} \cdot B_{11} \qquad +A_{12} \cdot B_{21}$$

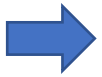
➡ Which is exactly how we need to compute C_{11}

Strassen's algorithm detail

- Similarly, $C_{12} = P_1 + P_2$

$$A_{11} \cdot B_{12} \cancel{- A_{11} \cdot B_{22}}$$
$$\cancel{A_{11} \cdot B_{22}} + A_{12} \cdot B_{22}$$

$$A_{11} \cdot B_{12} \quad + A_{12} \cdot B_{22}$$



This is how C_{12} needs to be computed

- $C_{21} = P_3 + P_4$

$$A_{21} \cdot B_{11} \cancel{+ A_{22} \cdot B_{11}}$$
$$\cancel{- A_{22} \cdot B_{11}} + A_{22} \cdot B_{21}$$

$$A_{21} \cdot B_{11} \quad + A_{22} \cdot B_{21}$$

This is how C_{21} needs to be computed

Strassen's algorithm detail

- $C_{22} = P_5 + P_1 - P_3 - P_7$

$$
\begin{array}{llllll}
\cancel{A_{11} \cdot B_{11}} & \cancel{+A_{11} \cdot B_{22}} & \cancel{+A_{22} \cdot B_{11}} & +A_{22} \cdot B_{22} & & \\
 & \cancel{-A_{11} \cdot B_{22}} & & & \cancel{+A_{11} \cdot B_{12}} & \\
 & & \cancel{-A_{22} \cdot B_{11}} & & & \cancel{-A_{21} \cdot B_{11}} \\
\cancel{-A_{11} \cdot B_{11}} & & & & \cancel{-A_{11} \cdot B_{12}} & \cancel{+A_{21} \cdot B_{11}} \quad +A_{21} \cdot B_{12} \\
\hline
 & & A_{22} \cdot B_{22} & & & +A_{21} \cdot B_{12}
\end{array}
$$

➡ This is how C_{22} needs to be computed.

Analysis of Strassen's algorithm

- Base case: *n* == 1, perform a scalar multiplication, just as in line 4 of SQUARE-MATRIX-MULTIPLY-RECURSIVE.

- For $n > 1$, steps 1, 2, and 4 take a total $\Theta(n^2)$ time.
- Step 3 performs **seven** multiplications of $n/2 \times n/2$ matrices, i.e., seven subproblems.
  - Hence, the recurrence for Strassen's algorithm is:

$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1 \\ 7T(n/2) + \Theta(n^2), & \text{if } n > 1 \end{cases}$$

- By master method, the solution is $T(n) = \Theta\left(n^{\log_b a}\right) = \Theta\left(n^{\log_2 7}\right) = \Theta(n^{2.81})$
- Which is asymptotically faster than $\Theta(n^3)$

Does a quadratic-time algorithm exist?

❑ Asymptotic complexities:

- $O(n^3)$, naïve approach
- $O(n^{2.808})$, Strassen (1969)
- $O(n^{2.796})$, Pan (1978)
- $O(n^{2.522})$, Schonhage (1981)
- $O(n^{2.517})$, Romani (1982)
- $O(n^{2.496})$, Coppersmith and Winograd (1982)

…

…

- $O(n^{2.3728642})$, V. Williams (2011)
- $O(n^{2.3728639})$, Le Gall (2014)

Example

Problem

- Show how to multiply the complex numbers $a + bi$ and $c + di$ using only **three multiplications** of real numbers. The algorithm should take a, b, c, and d as input and produce the real component $ac - bd$ and the imaginary component $ad + bc$ separately.

- Hints: Consider Strassen's approach
  - 1. Find three multiplications, e.g., A, B, and C
  - 2. Compute the real and imaginary components using A, B, and C